Estrategia de mitigación para RLS remota y Realtime

Proyecto: Gestor financiero · Portafolio Intel

Fecha: 27 de agosto de 2026

Estado: estrategia de cierre y especificación. No se han aplicado migraciones, cambios de publicación, políticas, claves, conectores ni operaciones sobre el proyecto remoto.

Alcance: cerrar los hallazgos pendientes de aislamiento por organización, privilegios, Realtime y consistencia de la UI persistente.

> **Objetivo de seguridad:** un cliente autenticado solo puede descubrir, leer, modificar o recibir por Realtime aquello que corresponde a su pertenencia y rol actuales. Un evento de datos solo puede invalidar el dominio mínimo necesario y una respuesta antigua nunca puede sustituir una instantánea más reciente.

Índice

1. Hallazgos y riesgo actual
2. Principios de cierre
3. Mitigación RLS paso a paso
4. Mitigación Realtime paso a paso
5. Diseño de código recomendado
6. Plan de pruebas de seguridad y carga
7. Secuencia de despliegue y reversión
8. Criterios para declarar cierre
9. Referencias

1. Hallazgos y riesgo actual

La aplicación carga datos del espacio autenticado mediante consultas separadas de cartera, snapshots, posiciones, alertas, fuentes, transacciones, análisis y reglas. En el código actual, una única suscripción se registra sobre seis tablas: `portfolio_snapshots`, `market_quotes`, `alerts`, `data_sources`, `analysis_runs` y `alert_rules`. Cada callback invoca `loadWorkspace()`, que vuelve a consultar siete conjuntos de datos. Esto es correcto como primera versión funcional, pero una ráfaga de cambios puede provocar solicitudes

redundantes, renders intermedios y una carrera donde una respuesta antigua llegue después de una nueva.

Las políticas y helpers están declarados localmente y existe evidencia de RLS en el modelo, pero el cierre de seguridad exige probar el **estado efectivo remoto** después de cada migración: RLS habilitado, grants reales, policies de cada operación, vistas, funciones, publicación Realtime y comportamiento con roles/organizaciones separados. La documentación de Supabase es clara: los grants controlan si un rol puede intentar una operación y las policies restringen qué filas puede afectar; añadir una policy no revoca grants existentes. [1]

Área	Riesgo pendiente	Consecuencia que se debe impedir
RLS	Policy correcta pero grants demasiado amplios, vista `security definer`, helper expuesto o tabla nueva sin RLS.	Lectura/escritura accidental entre organizaciones o por rol `anon`.
RLS	Controles `using` sin `with check`, FKs que no vinculan organización, o DML desde cliente sobre derivados.	Un editor reasigna la fila a otra organización/cartera o corrompe cálculos.
Realtime	Eventos de seis tablas llaman a una carga total de siete datasets.	Tormenta de consultas, latencia, consumo y estado visual inestable.
Realtime	Evento para una fila concreta se trata como actualización global.	Carga y exposición de datos innecesarios.
Realtime	Respuesta obsoleta gana la carrera o la membresía cambia sin renovar token/canal.	Dashboard incoherente o permisos que tardan en reflejarse.
Observabilidad	No existe evidencia reproducible de allow/deny, publicación o volumen de eventos.	No se puede demostrar que la remediación cerró el hallazgo.

---

2. Principios de cierre

Principio	Decisión de diseño
Denegar por defecto	`anon` no recibe ningún privilegio para datos de cartera. `authenticated` recibe únicamente el mínimo DML que realmente ofrece la interfaz.
RLS + grants, no RLS sola	Toda tabla expuesta se protege con ambos controles en la misma migración. Cada operación se prueba de forma independiente.
Aislamiento compuesto	Las filas de cartera incluyen `organization_id` y `portfolio_id`; las relaciones derivadas validan ambos identificadores, no solo uno.
Derivados sin DML del navegador	Ledger, valoraciones normalizadas y resultados TWR/XIRR/bridge se generan en backend controlado. El cliente pide un recálculo, nunca inserta resultados.
Datos mínimos por evento	La suscripción notifica una invalidez por dominio, no transporta el dataset ni dispara una recarga total.
Última versión gana	Toda carga usa una generación/versión y descarta respuestas que llegaron tarde.
Cambios de acceso revocan canal	Tras cambio de membresía/rol/policy, se refresca JWT, se elimina canal y se vuelve a cargar una instantánea autorizada.
Evidencia antes de producción	Migración, pgTAP, pruebas de dos organizaciones, Advisor, métricas y revisión humana son un único gate.

---

3. Mitigación RLS paso a paso

Paso 1. Congelar el contrato de acceso antes de editar SQL

Crear una matriz explícita por tabla, operación, rol y ruta de escritura. Un ejemplo de partida para el dominio de rendimiento se muestra abajo; debe completarse para **todas** las tablas públicas, incluidas las ya existentes y las nuevas.

Recurso	`anon`	`viewer` miembro	`editor` miembro	`owner` miembro	Proceso backend controlad
`transactions` , `positions` , `snapshots`	Sin acceso.	Lectura de su organización/cartera autorizada.	Lectura; DML solo si existe flujo de UI definido.	Igual o superior, según matriz.	Ingesta/coada.
`data_sources` , `sync_runs`	Sin acceso.	Lectura sin secretos.	Solicita/configura fuentes aprobadas según producto.	Gestiona revocación/propiedad.	Sincroniza registra resultado saneado.
`return_policies`	Sin acceso.	Lectura.	Crea una versión nueva; no altera cálculos históricos.	Retira/activa versiones.	Lectura.
`return_ledger_events` , `portfolio_valuations`	Sin acceso.	Lectura de auditoría necesaria.	Sin DML directo desde navegador.	Sin DML directo desde navegador.	Inserta/noza de form idempoter
`return_calculation_runs` , `return_attribution_*`	Sin acceso.	Lectura.	Solicita cálculo, no crea líneas.	Igual, según matriz.	Genera resultados iones.
`audit_events`	Sin acceso.	Lectura limitada si la UX la necesita.	Sin inserción de eventos arbitrarios.	Lectura administrativa, no alteración.	Inserta eventos de auditoría.

Ninguna pantalla debe añadir privilegios por comodidad. Si el producto no expone una operación, no existe grant ni policy para ella. Supabase recomienda definir policies por operación (`select` , `insert` , `update` , `delete`) y rol, en lugar de una policy opaca `for all` . [1]

Paso 2. Inventariar el estado remoto efectivo en staging

Con una cuenta administrativa de staging y sin copiar secretos a documentos, ejecutar una auditoría de solo lectura que recoja:

Inventario	Evidencia requerida	Condición de aceptación

RLS de tablas	`pg_class.relrowsecurity` por tabla expuesta.	Todas las tablas de `public` que llegan por API tienen RLS habilitada.
Grants	`information_schema.role_table_grants` y privilegios por role.	`anon` no tiene DML ni lectura de datos privados; `authenticated` solo lo autorizado.
Policies	`pg_policies` con `cmd`, `roles`, `qual`, `with_check`.	Cobertura de cada operación requerida, sin policy permisiva accidental.
Vistas	Definición, schema, `security_invoker` /owner y grants.	Ninguna vista pública elude RLS de tablas privadas.
Funciones	`proconfig`, `prosecdef`, `search_path`, `proacl`.	Ninguna función con privilegios elevados permite cruzar organizaciones.
Publicaciones	Tablas de `supabase_realtime` y `REPLICA IDENTITY`.	Solo tablas necesarias; no se publica información cruda por defecto.
Índices	Índices de filtros de policy y consultas.	`organization_id`, `portfolio_id` y miembros soportan el patrón de acceso real.

No se debe confiar en la definición JSON local como prueba de que el proyecto remoto coincide. El resultado del inventario, con nombres de objetos y sin datos privados, se adjunta a la revisión de cambio.

Paso 3. Rehacer grants y policies de cada tabla en una migración reproducible

Para cada tabla expuesta, la migración sigue esta secuencia: activar RLS, revocar el acceso por defecto de `anon` y `authenticated`, conceder únicamente las operaciones necesarias y crear una policy por operación. Supabase documenta el mismo orden y recuerda que una policy no elimina grants preexistentes. [1]

SQL

```sql
-- Patrón de tabla de solo lectura para clientes autenticados.
alter table public.return_calculation_runs enable row level security;

revoke all on table public.return_calculation_runs from anon, authenticated;
```

```sql
grant select on table public.return_calculation_runs to authenticated;

create policy return_runs_select_member
on public.return_calculation_runs
for select
to authenticated
using (
  private.is_organization_member(organization_id)
);

-- No existe policy ni grant de INSERT / UPDATE / DELETE al cliente.
-- La generación se ejecuta exclusivamente en backend controlado.
```

Cuando la interfaz puede editar una tabla —por ejemplo, una configuración no derivada— se crean policies separadas. La fila previa y la nueva fila se validan mediante `using` y `with check`; no se permite cambiar el `organization_id` o la cartera a otra que no autorice el rol.

SQL

```sql
alter table public.return_policies enable row level security;
revoke all on table public.return_policies from anon, authenticated;
grant select, insert on table public.return_policies to authenticated;

create policy return_policies_select_member
on public.return_policies
for select
to authenticated
using (private.is_organization_member(organization_id));

create policy return_policies_insert_editor
on public.return_policies
for insert
to authenticated
with check (
  private.is_organization_editor(organization_id)
  and created_by = (select auth.uid())
);

-- No UPDATE de una versión usada: la corrección se modela como versión nueva.
```

Nota de optimización: revisar que los helpers `private.is_organization_member` y `private.is_organization_editor` conservan `search_path` fijado, owner restringido y grants de ejecución mínimos. En policies donde el helper no depende de la fila, usar `select private.is_organization_member(organization_id)` solo si el plan y el helper lo permiten. Mantener índices que empiecen por columnas filtradas evita que RLS degrade lecturas a medida que las tablas crecen.[1]

Paso 4. Cerrar vías laterales: vistas, funciones, FKs y metadatos

1. Toda vista que un cliente pueda consultar se crea con `security_invoker = true` en Postgres 15+ o se mueve a un schema no expuesto y se consulta por un endpoint controlado. Las vistas pueden evitar RLS por defecto si se crean como `security definer`. 1
2. Toda función `security definer` revisa owner, `search_path`, parámetros, grants y operaciones que permite. Las funciones de consulta no aceptan un `organization_id` arbitrario si no prueban pertenencia dentro de la propia función.
3. Las nuevas tablas de ledger se enlazan mediante FK compuesta `(organization_id, portfolio_id)` para no poder asociar una cartera de otra organización por error. FKs de instrumento, fuente, sync run o evento también se comprueban con constraints o función controlada.
4. `connection_config` y campos de auditoría no guardan secretos, URL privadas, tokens o cabeceras. El ledger conserva ID de fuente y versión, no credenciales.
5. Las tablas de resultados derivados restringen DML de cliente. Una Edge Function o endpoint de servidor realiza validación de sesión, organización, cartera y payload mínimo antes de usar capacidad privilegiada. La `service_role` no se expone al navegador: RLS no la limita. 1

Paso 5. Escribir pgTAP antes de aplicar en producción

Ubicar pruebas en `supabase/tests/` y validar por tabla: grants, `anon`, miembro viewer, miembro editor, owner y no-miembro de otra organización. En una denegación por `USING`, la operación puede coincidir con cero filas sin lanzar error; la prueba debe verificar que la fila de destino permanece intacta. Para `WITH CHECK` o falta de grant, se espera `42501`. Supabase proporciona estas pautas y recomienda `RETURNING` en escrituras permitidas. 1

Ejemplo abreviado de patrón de test:

SQL

```sql
begin;
select plan(12);

-- Preparar dos organizaciones y sus cuentas ficticias.
-- Insertar un resultado de cálculo perteneciente solo a la organización A.

set local role anon;
select throws_ok(
  $$select * from public.return_calculation_runs$$,
  '42501', null,
  'anon no puede leer resultados'
```

```
);

set local role authenticated;
set local request.jwt.claim.sub = 'uuid-miembro-org-b';
select is_empty(
  $$select id from public.return_calculation_runs
    where id = 'uuid-run-org-a'$$,
  'un miembro de B no lee un resultado de A'
);

set local request.jwt.claim.sub = 'uuid-viewer-org-a';
select results_eq(
  $$select id from public.return_calculation_runs
    where id = 'uuid-run-org-a'$$,
  array['uuid-run-org-a'],
  'un viewer de A lee el resultado autorizado'
);

select * from finish();
rollback;
```

Paso 6. Verificar rendimiento de policy sin ampliar permisos

Antes de mover a producción, ejecutar planes de consulta representativos con un volumen sintético suficiente. Validar índices para `organization_id`, `portfolio_id`, membership y orden temporal. El Security Advisor y Performance Advisor de Supabase ayudan a detectar RLS mal configurada, FKs no indexadas o policies lentas; sus resultados son una señal de revisión, no un sustituto de pruebas de aislamiento. [2]

---

4. Mitigación Realtime paso a paso

Paso 1. Sustituir “recarga total por evento” por dominios de invalidación

El estado deja de tener un único `loadWorkspace()` como reacción a cualquier cambio. Se divide en loaders puros y dominios explícitos.

Tabla que cambia	Dominio a invalidar	Carga que sí se realiza	Carga que no debe ocurrir
`portfolio_snapshots`	`valuation`	Snapshots, KPI y serie visual de la cartera activa.	Alertas, fuentes, reglas, análisis y transacciones completas.

`market_quotes`	`valuation`	Posiciones/KPIs que dependan de quote, con throttle.	Fuentes, reglas o historial de transacciones.
`alerts`	`alerts`	Lista/badge de alertas.	Snapshots, fuentes, análisis o tabla de operaciones.
`data_sources`	`dataQuality`	Fuentes y sellos de frescura/cobertura.	Serie de valor, alertas o transacciones.
`analysis_runs`	`analysis`	Resultado de análisis visible.	Cartera completa si no hay dependencia real.
`alert_rules`	`rules`	Reglas y estado operacional.	Valoraciones, posiciones o informes.
`return_calculation_runs` futuro	`returns`	KPI de rendimiento y último bridge.	Ledger crudo o todos los datasets.

Cada dominio mantiene su propio `loading`, `lastAcceptedVersion`, `lastUpdatedAt`, `error` y `stale`. La UI conserva los últimos datos válidos mientras actualiza; nunca borra el workspace completo ante una notificación.

Paso 2. Agrupar eventos y cargar una vez por ventana

En lugar de llamar directamente a un loader en cada callback, el callback marca el dominio en un conjunto y programa una única ejecución en una ventana pequeña configurable. Es una coalescencia de invalidaciones, no una supresión de seguridad: la lectura posterior sigue siendo una consulta autorizada por RLS.

Plain Text

```
export type RealtimeDomain =
  | "valuation"
  | "alerts"
  | "dataQuality"
  | "analysis"
  | "rules"
  | "returns";

export type Invalidation = {
  domain: RealtimeDomain;
  organizationId: string;
  portfolioId: string | null;
  eventAt: string;
```

```
  eventVersion: string | number | null;
};
```

El buffer agrupa la unión de dominios recibidos. Si llegan diez eventos de `market_quotes` para la misma cartera dentro de la ventana, se realiza una sola carga `valuation`. Si llega un evento de alertas, añade `alerts` sin forzar el resto. Se registra el contador de eventos recibidos y cargas emitidas para demostrar que la coalescencia funciona.

Paso 3. Restringir eventos, tablas y filtros al mínimo

1. Escuchar solo tablas necesarias y solo eventos que cambian la vista. Si la UI no soporta eliminación física, preferir `INSERT` / `UPDATE` y soft delete con `deleted_at`; los filtros de Postgres Changes tienen limitaciones específicas con `DELETE` y la identidad de réplica. [3]
2. Para cada tabla con `organization_id`, usar filtro por organización; cuando la vista trabaja con una cartera concreta, filtrar también por `portfolio_id` si el patrón de filtro y el modelo de tabla lo permiten.
3. No añadir automáticamente al publication `supabase_realtime` tablas de ledger, cash flows o configuraciones sensibles. El dashboard normalmente solo necesita un marcador de cálculo actualizado y después cargar un resultado autorizado.
4. No usar payload de evento como fuente de verdad. El callback solo invalida; el loader recupera la instantánea autorizada mediante RLS.
5. Revisar `REPLICA IDENTITY FULL` únicamente cuando sea imprescindible recibir datos anteriores en DELETE/UPDATE. Tiene coste y no sustituye la política de borrado suave.

Supabase documenta que los filtros de Postgres Changes se evalúan en servidor y los eventos filtrados no salen de la base. [3] Aun así, los filtros son optimización y reducción de exposición; RLS sigue siendo el control principal de autorización.

Paso 4. Evitar carreras: versión de carga y aceptación condicional

Cada refresh obtiene una generación local monotónica y opcionalmente una versión de snapshot/calculation run. Al resolver una consulta, la respuesta se aplica solo si todavía pertenece a la generación más reciente de ese dominio y su versión no es anterior a la aceptada.

Plain Text

```
export type DomainLoadState<T> = {
  data: T | null;
  status: "idle" | "loading" | "ready" | "stale" | "error";
  requestedGeneration: number;
```

```
  acceptedGeneration: number;
  lastVersion: string | number | null;
  lastUpdatedAt: string | null;
  error: string | null;
};
```

Si una consulta iniciada antes acaba después de otra más nueva, se descarta sin renderizar. Un error de refresh tampoco borra el último contenido válido; marca `stale` y ofrece reintento controlado. Esta regla evita que una ráfaga retroceda los KPIs o la gráfica a un snapshot antiguo.

Paso 5. Cambios de membresía, JWT y canales privados

Para Postgres Changes con RLS, Supabase entrega cambios únicamente a clientes que pueden leer esos registros según las policies de la tabla.[4] Para Broadcast/Presence, la autorización se controla mediante policies sobre `realtime.messages` y un canal creado con `config: { private: true }`.[4]

El diseño recomienda dos patrones, nunca mezclados sin motivo:

Patrón	Uso correcto	Requisito de seguridad
Postgres Changes filtrado	Cambios poco frecuentes de tablas del dashboard.	RLS probada sobre tabla, publicación mínima, filtro de organización/cartera y callback que invalida.
Broadcast privado de invalidación	Señal mínima enviada por backend tras recalcular o completar un lote.	Topic mapeado a organización/cartera, policy `realtime.messages`, canal privado y payload sin datos financieros.

Cuando se cambia membresía, rol o policy, no basta con mantener el canal actual. La autorización de Realtime se calcula al unirse y se actualiza cuando recibe un JWT nuevo; si no recibe uno, el canal termina al expirar el token.[4] El flujo obligatorio es: invalidar sesión de UI, solicitar/recibir token actualizado, `removeChannel`, volver a suscribirse con topic autorizado y hacer refetch de una instantánea canónica. No se confía en un canal previamente autorizado.

Paso 6. Usar Broadcast privado para señales futuras de cálculo

Para `return_calculation_runs`, el backend puede enviar una notificación mínima —por ejemplo `{ domain: "returns", portfolioId, calculationRunId, version }` — a un topic privado y

autorizado. No incluye cifras, ledger, valores de mercado, notas ni datos de terceros. El cliente comprueba que la cartera activa coincide y llama al loader RLS protegido.

La policy de `realtime.messages` debe comprobar tanto el topic como la pertenencia actual. Es más seguro mantener una tabla de mapeo de topics a cartera/organización, no parsear indiscriminadamente un ID recibido desde el topic. Los topics son identificadores de ruteo, no la fuente de autorización.

SQL

```sql
-- Esquema conceptual; revisar y probar en staging antes de aplicar.
create policy portfolio_topic_subscribe
on realtime.messages
for select
to authenticated
using (
  realtime.messages.extension = 'broadcast'
  and exists (
    select 1
    from public.portfolio_realtime_topics t
    where t.topic = (select realtime.topic())
      and private.is_organization_member(t.organization_id)
  )
);
```

La tabla auxiliar de topics, si se crea, también tiene RLS, FK de organización/cartera, grants mínimos y pruebas de aislamiento. La policy no acepta cualquier topic cuyo texto "parezca" contener un UUID.

---

5. Diseño de código recomendado

5.1. Registro explícito de impacto por tabla

Plain Text

```
export const realtimeImpactByTable = {
  portfolio_snapshots: ["valuation"],
  market_quotes: ["valuation"],
  alerts: ["alerts"],
  data_sources: ["dataQuality"],
  analysis_runs: ["analysis"],
  alert_rules: ["rules"],
  return_calculation_runs: ["returns"],
} as const satisfies Record<string, readonly RealtimeDomain[]>;
```

```
export type RealtimeTable = keyof typeof realtimeImpactByTable;
```

Este registro es el único sitio que decide la invalidación. El callback de Realtime no conoce componentes ni llama a `loadWorkspace()`.

5.2. Coalescedor por dominio y generación

Plain Text

```
export type RefreshBatch = {
  domains: ReadonlySet<RealtimeDomain>;
  generation: number;
  reason: "initial" | "realtime" | "manual" | "reconnect";
};

export function createInvalidationScheduler(
  flush: (batch: RefreshBatch) => Promise<void>,
  waitMs: number,
) {
  let pending = new Set<RealtimeDomain>();
  let timer: ReturnType<typeof setTimeout> | null = null;
  let generation = 0;

  return (domains: readonly RealtimeDomain[], reason: RefreshBatch["reason"] =
    for (const domain of domains) pending.add(domain);
    if (timer) return;

    timer = setTimeout(() => {
      const batch: RefreshBatch = {
        domains: new Set(pending),
        generation: ++generation,
        reason,
      };
      pending = new Set();
      timer = null;
      void flush(batch);
    }, waitMs);
  };
}
```

La implementación real devuelve además `dispose()` para limpiar el timer al desmontar componente y no ejecutar una carga después de cerrar sesión. El `waitMs` se decide con benchmark y se documenta; no es una garantía de entrega de eventos. Ante reconexión se ejecuta siempre `refresh("reconnect")`.

5.3. Aplicación segura de una respuesta

Plain Text

```
export async function refreshDomain<T>(
  current: DomainLoadState<T>,
  nextGeneration: number,
  load: () => Promise<{ data: T; version: string | number | null }>,
): Promise<DomainLoadState<T>> {
  const requested: DomainLoadState<T> = {
    ...current,
    status: "loading",
    requestedGeneration: nextGeneration,
    error: null,
  };

  try {
    const result = await load();

    if (nextGeneration < requested.acceptedGeneration) return requested;
    if (
      typeof result.version === "number" &&
      typeof requested.lastVersion === "number" &&
      result.version < requested.lastVersion
    ) return requested;

    return {
      ...requested,
      data: result.data,
      status: "ready",
      acceptedGeneration: nextGeneration,
      lastVersion: result.version,
      lastUpdatedAt: new Date().toISOString(),
    };
  } catch (error) {
    return {
      ...requested,
      status: requested.data ? "stale" : "error",
      error: error instanceof Error ? error.message : "No se pudo actualizar el
    };
  }
}
```

El ejemplo ilustra la regla de aceptación; en React se protege contra actualizaciones tras desmontaje y se mantiene el `acceptedGeneration` más reciente en un `ref`. El loader debe seleccionar exclusivamente columnas necesarias y usar límites/paginación en cada dominio.

6. Plan de pruebas de seguridad y carga

6.1. Pruebas RLS en pgTAP y staging

Identificador	Escenario	Evidencia de cierre
RLS-001	`anon` intenta `select/insert/update/delete` sobre cada tabla privada.	Falta de grant / error `42501`, sin modificación.
RLS-002	`viewer` de organización A consulta resultado, ledger y bridge de A.	Solo filas de A autorizadas.
RLS-003	`viewer` de B intenta IDs conocidos de A.	Cero filas en lectura, cero filas actualizadas/borradas y fila A intacta.
RLS-004	`editor` de A intenta insertar/actualizar `organization_id` o `portfolio_id` de B.	Denegación por `with check` o constraint; no hay fila cruzada.
RLS-005	Cliente autenticado intenta DML sobre resultado derivado.	Sin grant/policy; solo backend controlado escribe.
RLS-006	Vista o función pública intenta leer datos de A como usuario B.	Ninguna fuga; `security_invoker` o schema privado comprobado.
RLS-007	Cambio de rol/membresía.	Nueva sesión/token pierde acceso después de refresh/reconexión.
RLS-008	Advisor y `EXPLAIN` sobre consultas con policy.	Sin alerta de tabla RLS deshabilitada; índices y plan revisados.

6.2. Pruebas de Realtime y carrera

Identificador	Escenario con datos ficticios	Evidencia de cierre

RT-001	Diez updates de `market_quotes` de una cartera activa dentro de la ventana.	Una única carga `valuation`; contador de callbacks > cargas emitidas.
RT-002	Update de `alerts`.	Solo se actualiza dominio `alerts`.
RT-003	Eventos simultáneos de snapshots, fuente y regla.	Un batch con tres dominios; ninguna carga global.
RT-004	Dos refreshes en que la respuesta antigua termina última.	UI conserva respuesta de mayor generación/versión.
RT-005	Evento de organización B en cliente A.	No se entrega o no coincide con filtro; no genera invalidez local.
RT-006	Usuario B fuerza topic/organización A.	No se puede unir al topic privado ni recibir broadcast.
RT-007	Revocación de pertenencia mientras hay canal activo.	Tras refresh de token y resubscribe, el cliente deja de recibir/leer datos.
RT-008	Reconexión de WebSocket.	Estado pasa por `conectando`, refetch autorizado y después `activo`; no muestra dato antiguo como actualizado.
RT-009	Error temporal de loader.	Datos previos quedan `stale`, no se vacían; se registra error seguro sin payload sensible.
RT-010	Delete necesario.	Funciona según `REPLICA IDENTITY` /soft delete elegidos y no amplía filtros ni exposición.

6.3. Indicadores de operación

Métrica	Objetivo de observación	Señal de regresión
Eventos recibidos / refresh emitidos	Medir efectividad de coalescencia.	Relación cercana a 1 bajo ráfaga repetida.
Duración de join de Realtime	Detectar policies de topic demasiado costosas.	Incremento sostenido o `TIMED_OUT`.

Cargas por dominio	Localizar sobrecarga y dependencias accidentales.	`alerts` provoca `valuation` sin relación justificada.
Respuestas descartadas por versión	Identificar carreras reales.	Aumento persistente: revisar debounce/loaders.
Denegaciones RLS	Detectar flujo UI que pide operación no permitida.	Picos tras una migración; revisar grants antes de relajar policy.
Cobertura de tests allow/deny	Demostrar alcance.	Tabla o operación sin caso de prueba.

No registrar valores de cartera, notas libres, tokens, cookies ni `connection_config` en logs o telemetría. Los identificadores se minimizan y se sujetan a retención definida.

7. Secuencia de despliegue y reversión

Fase	Acción	Gate obligatorio	Reversión
A. Preparación	Escribir matriz, migración y pgTAP; no cambiar producción.	Revisión de seguridad y producto.	No aplica.
B. Staging RLS	Aplicar migración en staging con fixtures de dos organizaciones.	PgTAP allow/deny, grants, vistas, funciones, Advisor.	Restaurar migración inversa previamente revisada.
C. Staging Realtime	Activar publicación mínima y dominios coalescidos detrás de flag.	RT-001 a RT-010, métricas y reconexión.	Desactivar flag y mantener carga manual/autorizada.
D. Canary interno	Usuarios internos ficticios/consentidos y monitorización.	Sin fuga, sin carrera, presupuesto de carga aceptado.	Desuscribir dominios nuevos; no borrar históricos.
E. Producción	Migración versionada y despliegue gradual.	Aprobación explícita y evidencia staging adjunta.	Migración inversa + flag off + auditoría de eventos.
F. Post-despliegue	Repetir inventario remoto y tests de smoke.	Informe de evidencia firmado/revisado.	Abrir incidente si hay discrepancia.

La migración inversa no debe desactivar RLS ni devolver grants amplios. Una reversión funcional preferible es desactivar la nueva ruta/feature flag y conservar el modo seguro anterior, no abrir acceso para “recuperar servicio”.

8. Criterios para declarar cierre

El hallazgo RLS puede cerrarse únicamente si todos los controles siguientes están respaldados por evidencia de staging y, tras despliegue, por verificación remota:

Control	RLS cerrado si…
Inventario	Todas las tablas, vistas, funciones, grants y policies expuestas están inventariadas.
Denegación por defecto	`anon` no tiene acceso a datos privados y `authenticated` solo conserva operaciones de UX aprobadas.
Aislamiento	Dos organizaciones y roles distintos prueban lectura/escritura/reasignación cruzada denegada.
Derivados	Cliente no puede modificar ledger, valoraciones normalizadas ni resultados de cálculo.
Vistas/funciones	No existen rutas `security definer` o grants que eludan la pertenencia.
Pruebas	PgTAP por tabla/operación pasa y está en CI.
Rendimiento	Índices y Advisor revisados sin problema bloqueante.

El hallazgo Realtime puede cerrarse únicamente si:

Control	Realtime cerrado si…
Alcance	Cada tabla/evento tiene dominio de invalidación justificado y filtros mínimos.
Eficiencia	Una ráfaga coalescida no dispara `loadWorkspace()` ni recargas no relacionadas.

Consistencia	El mecanismo de generación/versión evita que respuestas tardías retrocedan datos.
Autorización	Los clientes solo reciben Postgres Changes autorizados por RLS o Broadcast de topics privados autorizados.
Revocación	Un cambio de membresía fuerza renovación/reconexión y elimina acceso anterior.
Reconciliación	Reconexión hace refetch de instantánea canónica antes de declarar datos actualizados.
Evidencia	RT-001 a RT-010, métricas y prueba multiorganización pasan en staging.

9. Referencias

[1] Supabase Docs — Row Level Security
[2] Supabase Docs — Performance and Security Advisors
[3] Supabase Docs — Postgres Changes
[4] Supabase Docs — Realtime Authorization
[5] Gestor financiero — hoja de ruta de seguridad y producto
[6] Gestor financiero — especificación de pruebas y esquema de retornos

Divulgación. La estrategia está fundamentada en documentación oficial de Supabase consultada el 27 de agosto de 2026 y en revisión de código local. No acredita por sí sola la configuración vigente en remoto, por lo que no se considera un cierre hasta superar migración, pruebas, monitorización y revisión en staging/producción. No implica asesoramiento financiero ni activa funcionalidades sobre datos reales.